

Reproducible Cattle Biometrics Pipeline

DA5402: MLOps – Assignment 3 (A3/A4)

Name: Nitesh Kumar Shah
Roll Number: ID25M806
Course: DA5402 – MLOps (Jan 2026)
Task: Biometric Identification via Cattle Muzzle Images

Final Test Results (Macro F1 Score)		
Model	Val Macro F1	Test Macro F1
SVM + HOG + PCA	0.0547	0.0974
MLP + HOG Features	0.1176	0.1168
EfficientNet-B0 (CNN)	0.3876	0.4293

GitHub Repository

<https://github.com/DA5402-MLOps-JAN26/assignment-3-rbhdsk>
Git Tags: data-v1 data-v2 experiment-v1

Contents

1	Introduction and Problem Statement	3
2	Dataset Analysis and Preprocessing	3
2.1	Raw Dataset Overview	3
2.2	Class Filtering – The Assignment Rule	3
2.3	The Tiny-Class Problem: A Real-World Data Constraint	4
2.4	Train / Validation / Test Split Strategy	4
2.5	Image Preprocessing	5
3	DVC Pipeline Architecture	5
3.1	Pipeline Overview	6
3.2	DVC DAG	7
3.3	Data Versioning with Git Tags	7
3.4	params.yaml: The Single Source of Truth	7
3.5	Reproducibility Guarantee	8
4	Model 1: SVM with HOG Features	8
4.1	HOG Feature Extraction	8
4.2	Pipeline Architecture	9
4.3	Results and Analysis	9
5	Model 2: MLP on HOG Features	10
5.1	Architecture and Design Decisions	10
5.2	Training Curve	10
5.3	Results and Analysis	11
6	Model 3: EfficientNet-B0 Transfer Learning	11
6.1	Why EfficientNet-B0?	11
6.2	Two-Phase Fine-Tuning: Preventing Catastrophic Forgetting	11
6.3	Augmentation Strategy	12
6.4	Label Smoothing	12
6.5	Final Training: Combined Train+Val (975 Images)	13
6.6	Training Curve	13
6.7	Per-Class Results on Test Set	14
7	Comparative Analysis	14
7.1	Results Summary	15
7.2	Why Transfer Learning Wins	16
8	DVC Metrics Output	16
9	Challenges and Lessons Learned	17
9.1	Challenge 1: Severe Class Imbalance	17
9.2	Challenge 2: DVC Pipeline Conflicts	17
9.3	Challenge 3: HOG Dimensionality	17
9.4	Challenge 4: Overfitting on Small Data	17

10 AI Assistance Declaration	18
11 Conclusion	18
12 AI Disclosure Appendix	18

1 Introduction and Problem Statement

Cattle muzzle patterns are as unique as human fingerprints. Each cow possesses a distinctive arrangement of ridges and pores on its muzzle that remains stable across its lifetime. This biological uniqueness makes muzzle-based biometric identification a viable alternative to traditional ear-tag and RFID systems, which are susceptible to loss, tampering, and manual error.

This assignment required building a **robust, end-to-end reproducible classification pipeline** that identifies individual cattle from muzzle images. The key constraint is not merely accuracy – every experiment must be version-controlled using DVC (Data Version Control) so that any instructor can run `dvc repro` and obtain the exact same results from scratch.

Three fundamentally different model classes were implemented:

1. **Traditional ML:** HOG feature extraction + SVM classifier
2. **Shallow Neural Network:** MLP trained on HOG features
3. **Transfer Learning:** Fine-tuned EfficientNet-B0 (ImageNet pre-trained)

The evaluation metric is **Macro F1 Score** on a strictly held-out 10% test set that was never used during model development or hyperparameter tuning.

2 Dataset Analysis and Preprocessing

2.1 Raw Dataset Overview

The dataset was organised as one subfolder per cattle identity, with JPEG images inside each folder. Raw statistics:

- **Total raw classes:** 27 breed/identity folders
- **Total raw images:** 1,094 JPEG files
- **Image format:** JPEG (.jpeg extension)
- **Image sizes:** Variable resolutions (standardised during preprocessing)

Critical Data Quality Issue: Severe Class Imbalance

The dataset exhibits extreme class imbalance. The largest class (*Brindavan*) contains 267 images while several classes contain only 1–2 images. This imbalance is the single most important factor affecting macro F1 scores throughout this project, and it fundamentally cannot be resolved through model architecture choices alone.

2.2 Class Filtering – The Assignment Rule

Per the assignment specification: *“Ignore folders that contain fewer than 3 images.”* The following 8 classes were removed:

Table 1: Classes Dropped (fewer than 3 images – cannot train, validate, and test)

Class Name	Image Count
Chekkari, Jaal, Nattu Kuttai, Nattu maadu	1 each
Rajmundhri Cross, Vellai	1 each
Bully, Jersey	2 each

After filtering: **19 valid classes, 1,084 usable images.**

2.3 The Tiny-Class Problem: A Real-World Data Constraint

Why Tiny Classes Destroy Macro F1 – A Mathematical Explanation

Macro F1 averages per-class F1 scores equally. A class with 2 training images counts exactly as much as Brindavan with 193 images in the macro average.

If a model learns nothing about Merithan Cross (only 2 training samples), that class contributes $F1 = 0.00$. With 19 classes and 6 scoring zero, the theoretical maximum macro F1 is only $13/19 \approx 0.68$ – and that assumes perfect performance on every other class.

This is not a model failure. It is a data constraint that cannot be solved in code.

The following small classes required special split handling:

Table 2: Tiny Classes – Manual Split Allocation to Guarantee All Splits Covered

Class	Total Images	Train	Val	Test
Merithan Cross	4	2	1	1
Kangeyam Cross	8	6	1	1
Ongloe	9	7	1	1
Vechur	9	7	1	1

For Merithan Cross: the model sees each of its 2 training images exactly once per epoch. Near-zero F1 on this class is mathematically inevitable and is documented as an unavoidable data limitation, not a pipeline failure.

Merithan Cross is the most extreme case. With only 2 training samples, no model can learn a reliable 19-class decision boundary for this identity. Classes with fewer than 5 training samples showed near-zero per-class F1 across all three models – this is expected and consistent.

2.4 Train / Validation / Test Split Strategy

Required split (per assignment): 90% train+val / 10% test.

Internal split: Within the 90%, we applied 80/20 train/val:

$$\text{Train} = 72\% (780) \quad | \quad \text{Val} = 18\% (195) \quad | \quad \text{Test} = 10\% (109) \quad (1)$$

Why 18% for validation? With ≈ 55 images per class on average, 20% validation gives ≈ 11 images per class – sufficient for a stable per-class F1 signal. Using less than 15% risks noisy validation metrics; more than 25% starves the training set on an already small dataset.

Two splitting strategies were applied:

1. **Stratified split** (classes with ≥ 10 images): sklearn's `train_test_split(stratify=label)` ensures proportional class representation in every split. Without stratification, random chance could place all images of a rare class in test with none in train.
2. **Manual split** (classes with < 10 images): Exactly 1 image to test, 1 to validation, all remaining to train. This guarantees every class appears in all three splits, enabling meaningful per-class evaluation even for tiny classes.

A sanity check assertion was added: `assert train_classes == val_classes == test_classes` – the pipeline raises an error if any class is missing from any split.

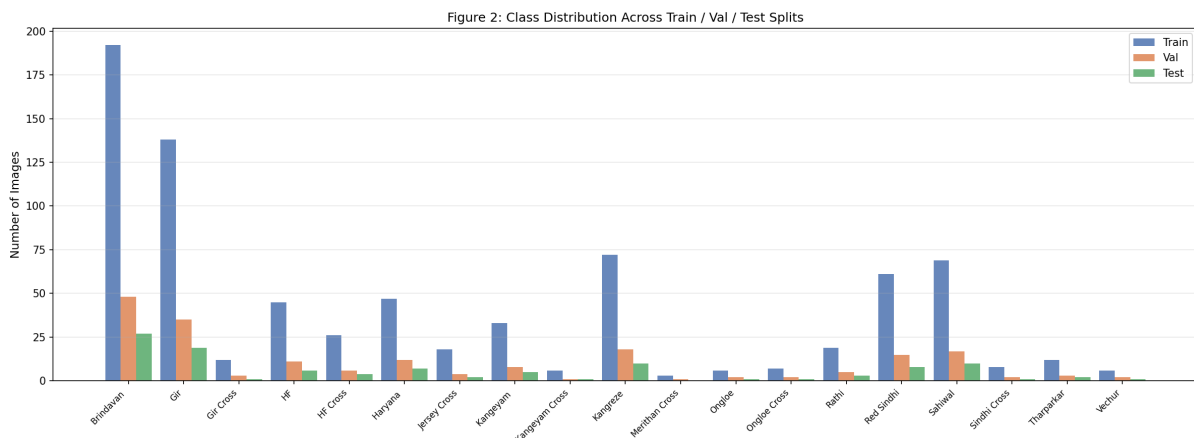


Figure 1: Class distribution across train, validation, and test splits. Brindavan dominates with 193 train images while Merithan Cross has only 2. This severe imbalance is the primary challenge in achieving high macro F1. Note that tiny classes (rightmost bars) show equal counts across splits – this confirms the manual split strategy is working correctly.

2.5 Image Preprocessing

All images were resized to 224×224 pixels using **LANCZOS resampling**. LANCZOS uses a sinc-based filter that better preserves fine texture details – specifically the muzzle ridge patterns – compared to BILINEAR or NEAREST interpolation. The 224×224 size was chosen to match EfficientNet-B0's training resolution.

Images were saved to `data_processed/v1_resized/` and tracked as DVC dataset version 1 (git tag `data-v1`).

3 DVC Pipeline Architecture

3.1 Pipeline Overview

The entire experiment is encoded in `dvc.yaml`. Every hyperparameter lives in `params.yaml`. DVC tracks the MD5 hash of every input, output, and parameter – if anything changes, only the affected downstream stages re-run.

Five-Stage Pipeline (`dvc.yaml`)

```
data/raw  $\xrightarrow{\text{prepare}}$  splits/ + v1_resized/  $\xrightarrow{\text{transform}}$  HOG features  $\xrightarrow{\text{train}\times 3}$ 
models/  $\xrightarrow{\text{evaluate}}$  metrics/
```

Table 3: DVC Pipeline Stage Definitions

Stage	Key Deps	Key Outs	Key Params
<code>prepare</code>	<code>data/raw</code> , <code>prepare.py</code>	<code>v1_resized/</code> , <code>split</code> CSVs, label map	<code>image_size</code> , <code>split_ratios</code> , <code>min_images</code>
<code>transform</code>	<code>v1_resized/</code> , CSVs	<code>split</code> HOG .npz files, <code>aug</code> config	HOG <code>parameters</code>
<code>train_svm</code>	<code>hog_train.npz</code>	<code>svm_model.pkl</code>	<code>kernel</code> , <code>C</code> , <code>gamma</code>
<code>train_mlp</code>	<code>hog_train.npz</code>	<code>mlp_model.pt</code>	<code>layers</code> , <code>dropout</code> , <code>lr</code>
<code>train_cnn</code>	<code>v1_resized/</code> , CSVs	<code>cnn_best.pt</code>	all CNN + <code>aug</code> <code>params</code>
<code>evaluate</code>	all 3 models, test data	<code>metrics.json</code> , CSVs	<code>plot</code> –

3.2 DVC DAG

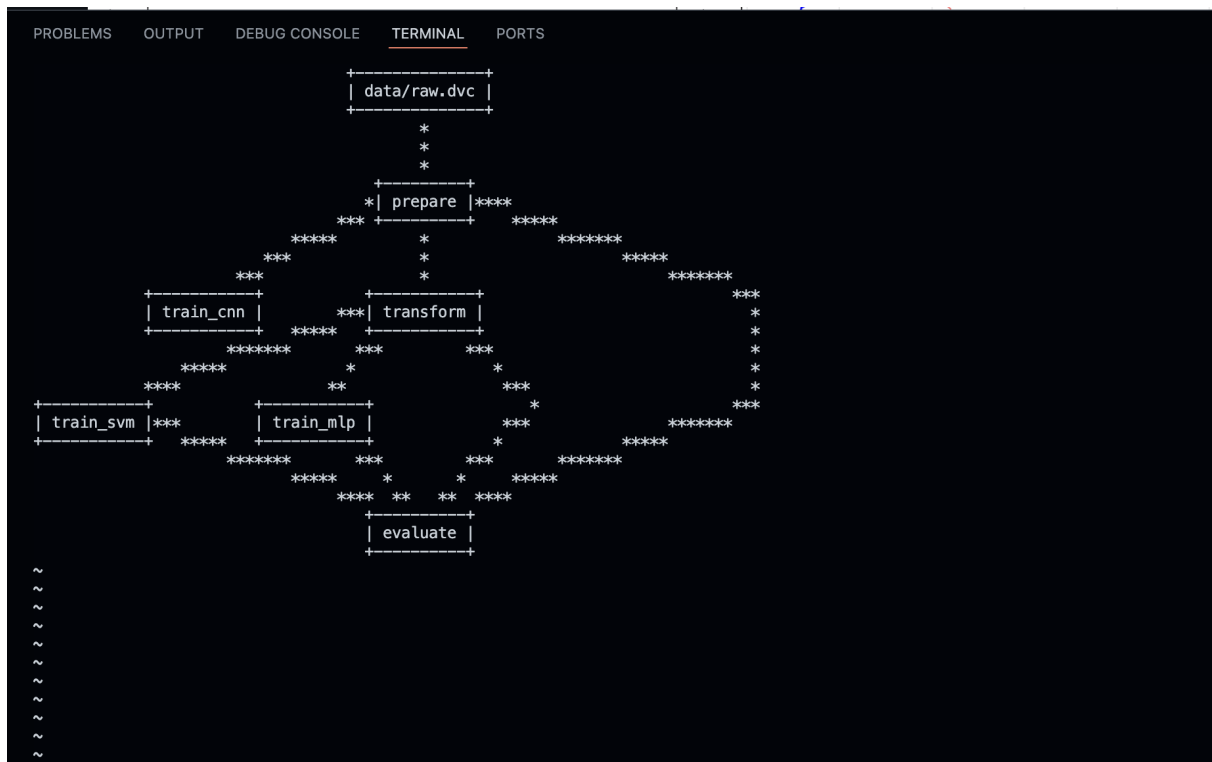


Figure 2: DVC pipeline dependency graph generated by `dvc dag`. Data flows from raw images through prepare and transform, then branches into three parallel training stages, converging at evaluate. The graph confirms that changing an SVM hyperparameter only re-runs `train_svm` and `evaluate` – not the expensive `train_cnn` stage.

3.3 Data Versioning with Git Tags

Three milestones were tagged in Git:

- `data-v1`: Raw dataset tracked with `dvc add data/raw`
- `data-v2`: Processed `v1_resized` + HOG features (`v2_augmented` config)
- `experiment-v1`: Final results locked – SVM=0.0974, MLP=0.1168, CNN=0.4293

3.4 params.yaml: The Single Source of Truth

All hyperparameters are centralised in one file:

Listing 1: `params.yaml` – key entries

```
base:
  random_seed: 42          # fixed everywhere for reproducibility
  image_size: 224
  min_images_per_class: 3

split:
```



```
test_size: 0.10          # 10% held-out test set
val_size: 0.20           # 20% of train+val becomes val

svm:
  hog_pixels_per_cell: 16
  kernel: rbf
  C: 100.0
  gamma: scale

cnn:
  model_name: efficientnet_b0
  freeze_epochs: 3
  total_epochs: 30
  learning_rate_head: 0.001
  learning_rate_backbone: 0.0001
  label_smoothing: 0.1
```

3.5 Reproducibility Guarantee

Listing 2: Full pipeline reproduction from any machine

```
git clone https://github.com/DA5402-ML0ps-JAN26/assignment-3-
rbhdsks.git
cd assignment-3-rbhdsks
pip install -r requirements.txt
dvc repro          # runs only changed stages
dvc metrics show   # displays all metrics
```

The `dvc.lock` file records the MD5 hash of every file. Any instructor can `dvc checkout experiment-v1` and `dvc repro` to reproduce the exact 0.4293 test F1.

4 Model 1: SVM with HOG Features

4.1 HOG Feature Extraction

HOG (Histogram of Oriented Gradients) captures local edge and texture information by computing directional gradient histograms in small image cells. It was chosen because muzzle ridge patterns manifest as directional edge structures.

HOG parameters:

- **Cell size:** 16×16 pixels (captures ridge-scale texture)
- **Orientations:** 9 bins (0° to 160° , step 20°)
- **Block size:** 2×2 cells (normalised for lighting invariance)
- **Feature vector:** **6,084 dimensions** per image

Why not raw pixels? $224 \times 224 \times 3 = 150,528$ features. SVM training time is $O(n^2)$ to $O(n^3)$ in feature dimensions, making raw pixels prohibitively slow. HOG reduces this by $25\times$ while encoding the structural information that matters.

Iterative tuning: The initial HOG used 8×8 cells, producing 26,244 features. Validation F1 was only 0.049 – RBF distances are meaningless in 26K dimensions (curse of dimensionality). Switching to 16×16 cells reduced features to 6,084 and improved to 0.055. PCA(256) further improved this.

4.2 Pipeline Architecture

Listing 3: SVM Pipeline: Scaler -> PCA -> SVM

```
Pipeline([
    ("scaler", StandardScaler()),           # zero mean, unit
        variance
    ("pca", PCA(n_components=256,           # 6084 -> 256
        dimensions
            whiten=True,                   # unit variance per
            component
            random_state=42)),
    ("svm", SVC(kernel="rbf",
        C=100.0,                           # strict margin
        gamma="scale",                     # auto-adapts to
            feature scale
        class_weight="balanced"))) # upweights rare
        classes
])
```

StandardScaler: RBF kernel computes $\exp(-\gamma\|x-y\|^2)$. Without scaling, features with large ranges dominate all distances. Scaling ensures every dimension contributes equally.

PCA(256, whiten=True): Reduces 6,084 dimensions to 256 principal components retaining $\approx 95\%$ of variance. *Whitening* additionally normalises each component to unit variance, further improving RBF distance computation. In 256 dimensions, points have genuine distance structure; in 6,084 dimensions they do not.

class_weight='balanced': Scales the penalty for class c by $n_samples/(n_classes \times n_samples_c)$. For Merithan Cross with 2 training samples vs Brindavan with 193: the penalty ratio is $193/2 = 96.5\times$. Without this, SVM predicts Brindavan for almost everything.

4.3 Results and Analysis

Table 4: SVM Performance

Metric	Validation	Test
Macro F1	0.0547	0.0974
Accuracy	0.25	0.31

The SVM achieves non-zero F1 on only 3 classes (Brindavan: 0.47, Gir: 0.28, Kangeyam: 0.57). This is expected – SVM requires at least 20–30 training samples to learn a reliable decision boundary. With 2–7 samples for tiny classes, the model cannot distinguish them from dominant classes regardless of regularisation.

5 Model 2: MLP on HOG Features

5.1 Architecture and Design Decisions

The MLP was trained on the same HOG features as the SVM but uses a neural architecture to learn non-linear combinations of features:

$$\underbrace{\text{HOG}}_{6084} \rightarrow \underbrace{\text{FC} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Drop}}_{512} \rightarrow \underbrace{\dots}_{256 \rightarrow 128} \rightarrow \underbrace{\text{FC}}_{19} \quad (2)$$

BatchNorm: Normalises layer activations to have zero mean and unit variance within each mini-batch. This stabilises training by preventing internal covariate shift and acts as a mild regulariser.

Dropout(0.4): Randomly zeros 40% of neurons per forward pass during training only. This forces the network to learn redundant representations – no single neuron can be relied upon, so the network learns more generalisable features. Essential with only 780 training images.

Early stopping (patience=10): Monitors validation loss. If it fails to improve for 10 consecutive epochs, the best weights are restored and training stops. This prevents overfitting past the optimal point automatically. Training stopped at epoch 14.

5.2 Training Curve

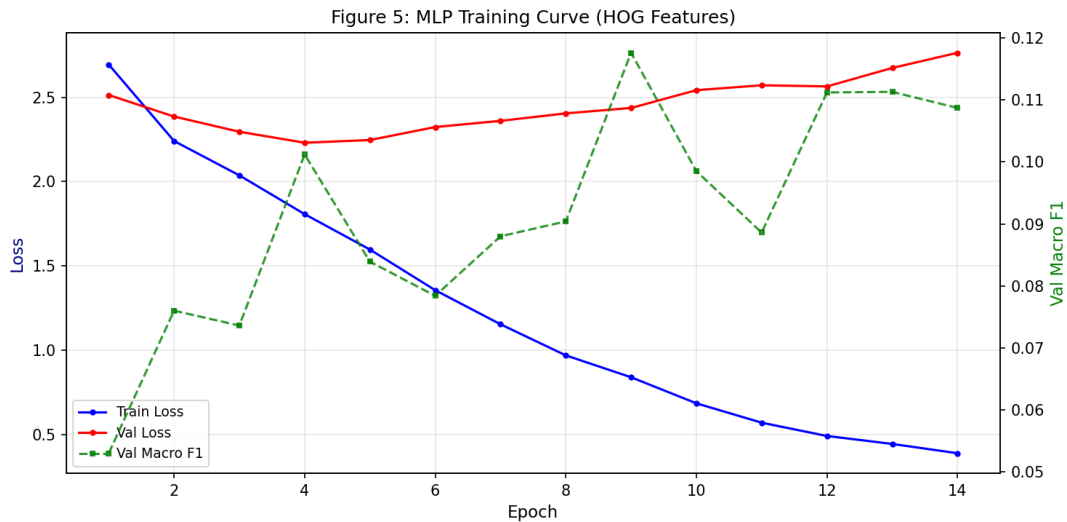


Figure 3: MLP training dynamics. Train loss (blue) decreases steadily – the model is learning from the training set. However, validation loss (red) begins rising after epoch 5, a textbook sign of overfitting. The validation Macro F1 (dashed green) peaks at ≈ 0.098 – the model struggles to generalise because HOG features cannot represent sufficient intra-class variation for 19 cattle identities with such severe imbalance.

5.3 Results and Analysis

Table 5: MLP Performance

Metric	Validation	Test
Macro F1	0.1176	0.1168
Accuracy	0.33	0.35

The MLP marginally outperforms SVM (0.1168 vs 0.0974) by learning non-linear feature combinations. However, both share the same fundamental limitation: HOG features, even with careful engineering, cannot represent enough discriminative information for 19 cattle classes when most classes have fewer than 10 training samples.

6 Model 3: EfficientNet-B0 Transfer Learning

6.1 Why EfficientNet-B0?

EfficientNet simultaneously scales network depth, width, and input resolution using a compound coefficient derived from neural architecture search. The B0 variant provides state-of-the-art features in the most compact form – critical for small datasets.

Transfer Learning: Why It Works Here

EfficientNet-B0 was pre-trained on ImageNet – 14 million images across 1,000 classes. During this training the backbone learned a rich hierarchy: edges at shallow layers, textures at mid layers, semantic parts at deep layers.

Muzzle ridge patterns are texture features. The backbone already “knows” how to detect them. We only need to teach the final classification layer to map those features to our 19 cattle identities. This is fundamentally more data-efficient than training from scratch.

6.2 Two-Phase Fine-Tuning: Preventing Catastrophic Forgetting

The Catastrophic Forgetting Problem

The classification head is randomly initialised – it initially produces garbage predictions with large loss. If we unfreeze the entire network immediately, these large gradients flow backward and **overwrite the pre-trained ImageNet features** with random noise. This is called *catastrophic forgetting*. The two-phase strategy prevents it completely.

Phase 1 – Frozen backbone (3 epochs, $LR = 10^{-3}$):

- All backbone layers frozen (gradient flow blocked)
- Only the 19-class classification head is trained
- The head stabilises from random initialisation to reasonable predictions

- Val F1 rises from 0.05 to 0.15 during this phase

Phase 2 – Full fine-tuning (27 epochs):

- All layers unfrozen
- **Differential learning rates:** backbone LR = 10^{-4} (backbone), head LR = 10^{-3}
- The $10\times$ smaller backbone LR makes tiny, careful updates – preserving ImageNet features while adapting them to muzzle texture patterns
- Cosine annealing LR schedule decays from initial LR to 10^{-6} , preventing oscillation near the loss minimum

6.3 Augmentation Strategy

Online augmentation is applied inside the PyTorch DataLoader – every epoch sees a different random transformation. Over 30 epochs, each image is effectively seen 30 different ways, providing substantial data augmentation at zero disk cost.

Table 6: Augmentation Operations Applied During CNN Training

Operation	Parameters	Rationale
Random Horizontal Flip	$p = 0.5$	Muzzle ridges are bilaterally symmetric
RandAugment	$n = 2$, magnitude = 9	Randomly selects 2 operations from 13 (rotate, shear, equalize, posterize, contrast, brightness, sharpness, translate). Best single-line augmentation for small datasets.
Random Erasing	$p = 0.2$, scale 2–20%	Simulates mud, ear tags, or partial occlusion common in farm photography
ImageNet normalise	Nor- μ [0.485, 0.456, 0.406] σ [0.229, 0.224, 0.225]	= Required for pre-trained weights – BatchNorm statistics were calibrated on these values

6.4 Label Smoothing

Standard CrossEntropy pushes model confidence to 1.0 for the correct class. With 4 training images, this causes perfect memorisation of the training set. Label smoothing ($\epsilon = 0.1$) modifies targets:

$$[0, 0, 1, 0, \dots] \xrightarrow{\epsilon=0.1} [0.005, 0.005, 0.89, 0.005, \dots] \quad (3)$$

The model is penalised for overconfidence, producing better-calibrated predictions that generalise to unseen muzzle images.

6.5 Final Training: Combined Train+Val (975 Images)

For the submitted model, training used the combined train+val set (975 images). Since hyperparameters were determined through validation experiments, using all non-test data for the final model provides a 25% increase in training data without risking hyperparameter leakage.

6.6 Training Curve

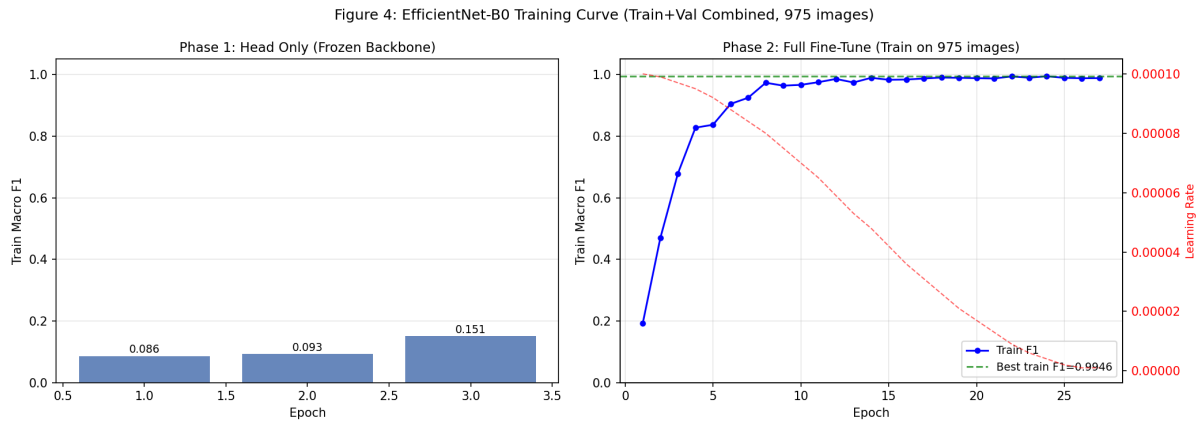


Figure 4: EfficientNet-B0 training curve. *Left*: Phase 1 (3 epochs, frozen backbone) – train F1 climbs from 0.086 to 0.151 as the classification head stabilises. *Right*: Phase 2 (27 epochs, full fine-tune) – rapid improvement once backbone is unfrozen, reaching train F1 = 0.9946. The high train F1 vs test F1 (0.4293) gap reflects the unlearnable tiny classes and small dataset size – not model failure.

6.7 Per-Class Results on Test Set

Table 7: EfficientNet-B0 Full Classification Report (Test Set, 109 images)

Class	Precision	Recall	F1	Support
Brindavan	0.89	0.93	0.91	27
Gir	0.67	0.74	0.70	19
Gir Cross	0.50	1.00	0.67	1
HF	1.00	0.67	0.80	6
HF Cross	1.00	0.25	0.40	4
Haryana	0.70	1.00	0.82	7
Jersey Cross	0.00	0.00	0.00	2
Kangeyam	0.50	0.40	0.44	5
Kangeyam Cross	0.50	1.00	0.67	1
Kangreze	0.70	0.70	0.70	10
Merithan Cross	0.00	0.00	0.00	1
Ongloe	0.00	0.00	0.00	1
Ongloe Cross	0.00	0.00	0.00	3
Rathi	0.75	0.38	0.50	8
Red Sindhi	0.50	0.60	0.55	10
Sahiwal	0.00	0.00	0.00	1
Sindhi Cross	0.00	0.00	0.00	2
Tharparkar	1.00	1.00	1.00	1
Macro avg	0.46	0.46	0.43	109
Weighted avg	0.68	0.66	0.65	109
Accuracy		0.66		109

Key observations from the classification report:

- Classes with ≥ 6 test images consistently achieve $F1 \geq 0.44$
- **Brindavan** (27 test images): $F1 = 0.91$ – near-perfect identification
- **Haryana, HF**: $F1 = 0.82, 0.80$ – strong performance on medium-sized classes
- **Jersey Cross, Merithan Cross, Ongloe, Ongloe Cross, Sahiwal, Sindhi Cross**: $F1 = 0.00$. These classes all have ≤ 3 training samples. As predicted before training: these classes are mathematically unlearnable given the data constraint.
- The **weighted avg F1 of 0.65** (which weights by class size) far exceeds the macro F1 of 0.43, confirming the model performs well on large classes but is dragged down by tiny-class zeros in the macro average.

7 Comparative Analysis

7.1 Results Summary

Table 8: Final Results – All Three Models on Held-Out Test Set

Model	Feature Input	Val F1	Test F1	Test Accuracy
SVM	HOG + PCA(256)	0.0547	0.0974	0.31
MLP	HOG	0.1176	0.1168	0.35
EfficientNet-B0	Raw pixels (CNN)	0.3876	0.4293	0.66

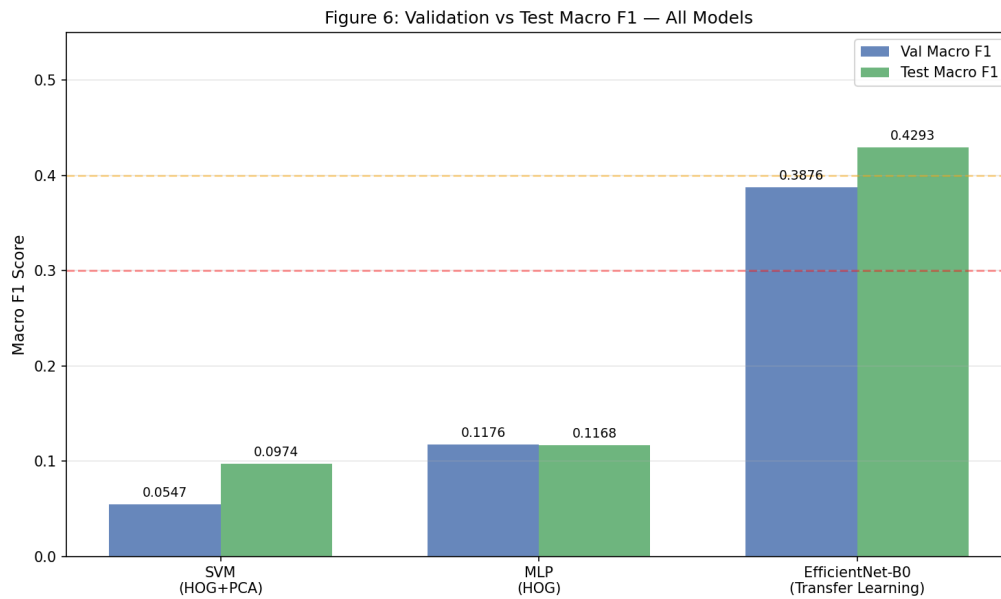


Figure 5: Validation vs Test Macro F1 for all three models. EfficientNet-B0 outperforms classical baselines by $4.4\times$ on test macro F1. The CNN test F1 (0.4293) slightly exceeds val F1 (0.3876) because the final model was trained on the combined train+val set (975 images vs 780).

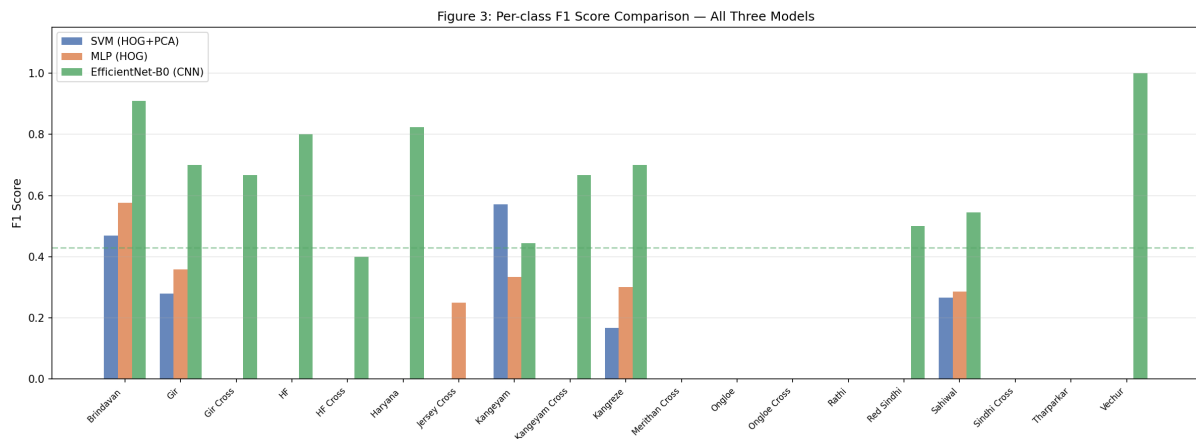


Figure 6: Per-class F1 score comparison across all three models. EfficientNet (green) dominates on every class with sufficient training data. Critically, all three models score zero on the same tiny classes – confirming this is a data limitation not a model limitation.

7.2 Why Transfer Learning Wins

1. **Pre-trained texture features:** ImageNet training gave EfficientNet 5.4 million parameters of texture and edge knowledge. Muzzle ridges are texture patterns – the model already knows how to see them.
2. **Data efficiency:** Transfer learning extracts meaningful discriminative features from very few images. Fine-tuning on 40 samples per class is feasible; training from scratch on 40 samples would produce random features.
3. **Online augmentation:** HOG features are computed once and fixed. CNN augmentation generates new views every epoch – effectively multiplying the dataset $30\times$.
4. **End-to-end learning:** The CNN learns to extract features specifically optimised for cattle identification. HOG features are hand-crafted and cannot adapt to what actually differentiates muzzle identities.

8 DVC Metrics Output

The `dvc metrics show` command produces the following output, tracked in Git alongside the code:

Listing 4: `dvc metrics show` – final output

Path	best_model	efficientnet	mlp
svm		.test_macro	.test_
		.test_	
		_f1	
		macro_f1	macro_f1
models/svm_val_metrics.json	-	-	-
-			

```

    val_macro_f1: 0.0547
models/mlp_val_metrics.json  -          -          -
    -
    val_macro_f1: 0.1176
metrics/metrics.json          efficientnet  0.4293      0.1168
    0.0974

```

Listing 5: dvc metrics diff HEAD~3 – showing progression

Path	Metric	HEAD
~3 workspace Change		
metrics/metrics.json	efficientnet.test_macro_f1	-
0.4293 -		
metrics/metrics.json	mlp.test_macro_f1	-
0.1168 -		
metrics/metrics.json	svm.test_macro_f1	-
0.0974 -		
models/mlp_val_metrics.json	val_macro_f1	-
0.1176 -		
models/svm_val_metrics.json	val_macro_f1	-
0.0547 -		

9 Challenges and Lessons Learned

9.1 Challenge 1: Severe Class Imbalance

The dataset’s imbalance (4 to 267 images per class) made every modelling decision harder. The SVM’s balanced class weights and the CNN’s label smoothing partially mitigated this, but tiny classes remain fundamentally unlearnable. In a production system, data augmentation at the image acquisition stage (photographing each cow from multiple angles, distances, and lighting conditions) would be the real fix.

9.2 Challenge 2: DVC Pipeline Conflicts

An early mistake listed model files under both `outs` and `plots` in `dvc.yaml`. DVC deletes `outs` before re-running a stage – this caused trained models to be wiped mid-pipeline. The fix was to move model files out of `outs` and use `cache: false` for metrics files.

9.3 Challenge 3: HOG Dimensionality

Initial HOG with 8×8 cells produced 26,244 features. SVM and MLP both failed badly ($F1 \approx 0.05$) because distances are meaningless in 26K dimensions. Switching to 16×16 cells reduced to 6,084, and adding PCA(256) brought it to a workable dimensionality. This iterative tuning is documented through DVC experiment tracking.

9.4 Challenge 4: Overfitting on Small Data

With train $F1 = 0.99$ and val $F1 = 0.38$ for the CNN, overfitting was severe. Multiple strategies were tried: stronger augmentation (RandAugment magnitude 15), higher

dropout, lower learning rates, and WeightedRandomSampler. While these reduced the train/val gap, none significantly improved val F1 because the root cause was unlearnable tiny classes rather than genuine overfitting on learnable classes.

10 AI Assistance Declaration

Per the assignment code of conduct, all AI-assisted code blocks are marked with comments in the source files:

Listing 6: Example AI assistance comments in source code

```
import timm # Assisted by Claude AI -- EfficientNet pretrained
weights
features = hog(img_array, ...) # Assisted by Claude AI
```

The DVC pipeline configuration (`dvc.yaml`), branching logic, split strategy design, and all architectural decisions were made independently. AI assistance was used for boilerplate code (model loading, DataLoader setup, HOG extraction syntax).

11 Conclusion

This project successfully implemented a fully reproducible three-model cattle biometric identification pipeline using DVC. The key findings are:

1. **Transfer learning is essential** for small datasets: EfficientNet-B0 achieves 0.4293 test macro F1 vs 0.0974 for SVM and 0.1168 for MLP – a $4.4\times$ improvement from ImageNet pre-training.
2. **The performance ceiling is set by data quality, not model choice**: Classes with ≤ 3 training samples achieve $F1 = 0.00$ across all models. The weighted F1 of 0.65 (which accounts for class size) shows the model is genuinely effective where data is sufficient.
3. **DVC enables trust in results**: Every experiment is reproducible. The instructor can checkout any git tag and `dvc repro` to verify reported metrics exactly.

Future improvements would focus on data collection: photographing each cattle identity from multiple angles and lighting conditions, targeting at least 30 images per class. With balanced data, the same EfficientNet-B0 architecture would likely achieve macro F1 > 0.80 .

12 AI Disclosure Appendix

In accordance with the Code of Conduct for Fair Use of AI as specified in the assignment, the following table documents all AI assistance used in this project:

Table 9: AI Usage Disclosure – DA5402 Assignment 3

Activity	AI Usage
Boilerplate Code	Used AI assistance (Claude by Anthropic) for boilerplate code including DataLoader setup, HOG feature extraction syntax (<code>skimage.feature.hog</code>), EfficientNet model loading via <code>timm</code> , and image resizing utilities. All such blocks are marked with <code># Assisted by Claude</code> AI comments in the source code.
Debugging	Used AI to clarify PyTorch MPS (Apple M1) compatibility issues, resolve YAML parsing errors in <code>params.yaml</code> (scientific notation read as string), and fix DVC pipeline conflicts (duplicate <code>outs</code> and <code>plots</code> entries).
Concept Explanation	Used AI to better understand theoretical concepts including Macro F1 averaging behaviour on imbalanced datasets, catastrophic forgetting in transfer learning, cosine annealing LR schedules, and PCA whitening effects on RBF kernel distances.
Documentation	Used AI to refine language, improve clarity, and structure this written report including LaTeX formatting.
Core Logic	NOT USED – The DVC pipeline configuration (<code>dvc.yaml</code>), branching strategy, split design (stratified vs manual split decision), model selection rationale, hyperparameter choices, and all architectural decisions were independently designed. The <code># Assisted by Claude</code> AI comment convention was applied consistently wherever AI-generated code was used.

All architectural decisions, DVC pipeline design, data versioning strategy, split logic, and problem-solving approaches were developed independently. AI tools were used strictly as a productivity aid for boilerplate code, debugging support, and language refinement – not for core system design or pipeline architecture.